ФСП КАРЬЕРНАЯ АРЕНА

ОФИЦИАЛЬНЫЙ АРХИТЕКТУРНЫЙ МАНИФЕСТ И ПРОЕКТНАЯ ДОКУМЕНТАЦИЯ

Регламент платформы: Архитектура, Алгоритмы, Тестирование, 152-ФЗ, Интеграции, Деплой

Команда разработки: «NO PHP — NO PROBLEMS — 2026» | Хакатон ФСП РФ 2026

Сопроводительная проектная документация

Цифровая платформа обратного найма с верификацией ФСП

> Команда: «NO PHP - NO PROBLEMS - 2026»

> Хакатон: Федерация спортивного программирования России (ФСП РФ) - 2026

> Версия документа: 1.0 | 10 октября 2026 г.

Оглавление

1. [Архитектура решения](#1-архитектура-решения)
2. [Механика обратного найма и ранжирования](#2-механика-обратного-найма-и-ранжирования)
3. [Механика тестирования и защиты от «слива»](#3-механика-тестирования-и-защиты-от-слива)
4. [Схема интеграции с ФСП ID](#4-схема-интеграции-с-фсп-id)
5. [Спецификация API и безопасность](#5-спецификация-api-и-безопасность)
6. [Инструкция по развёртыванию](#6-инструкция-по-развёртыванию)
7. [Команда проекта](#7-команда-проекта)

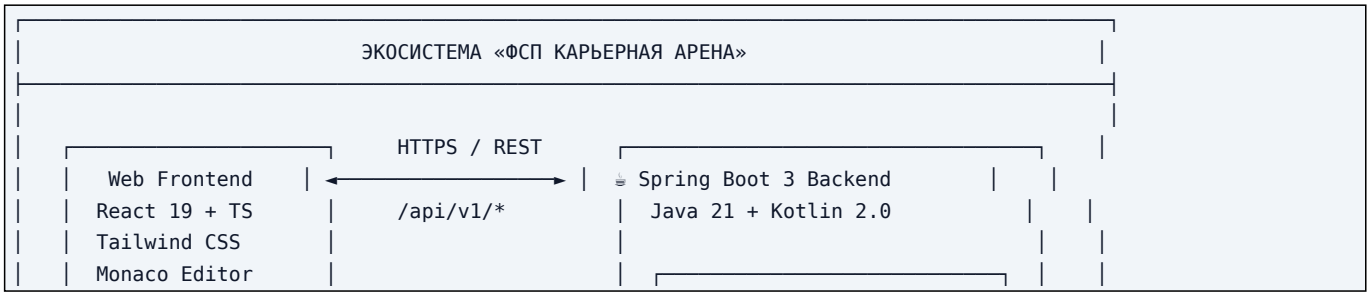
---

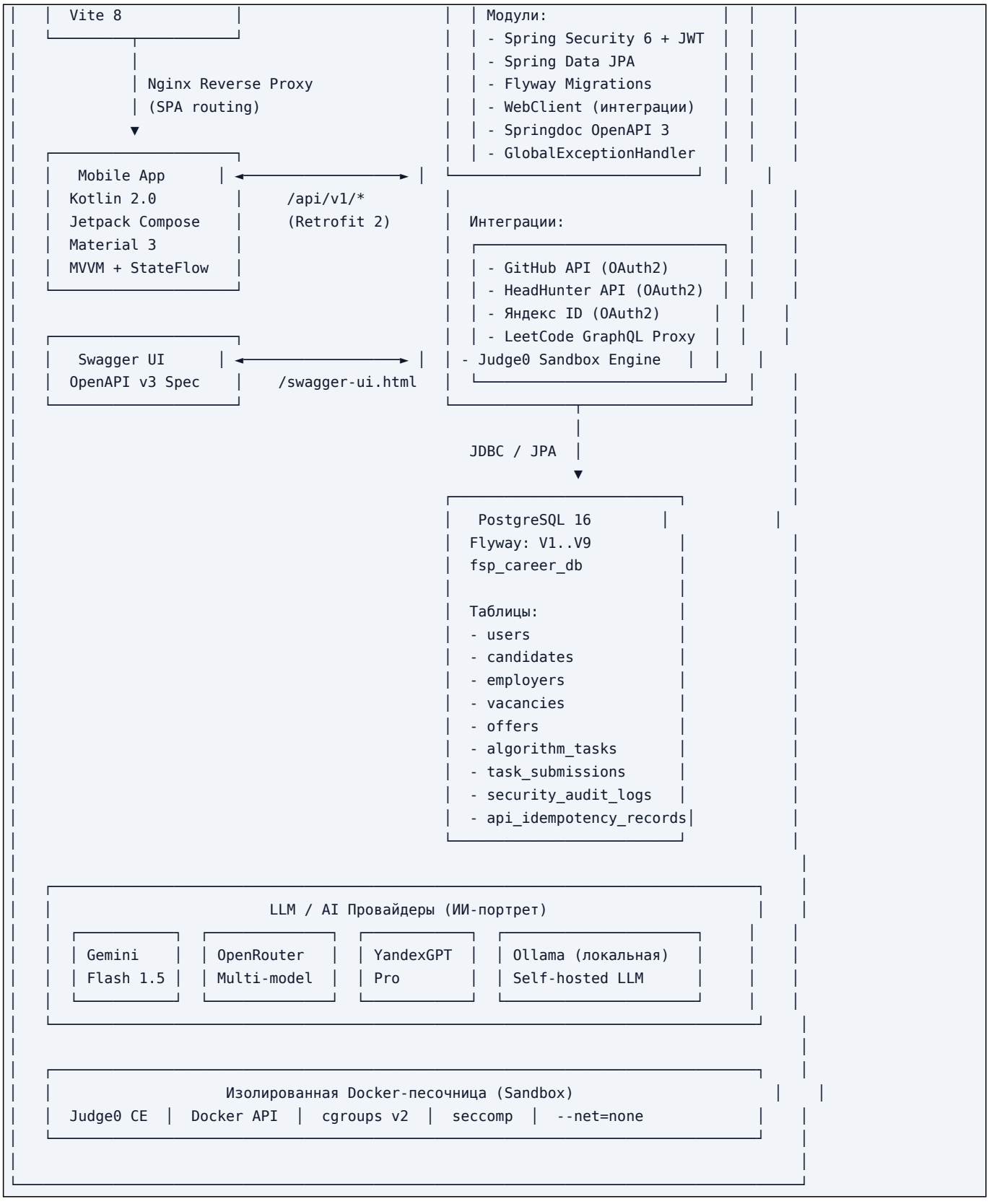
1. Архитектура решения

1.1. Общее описание

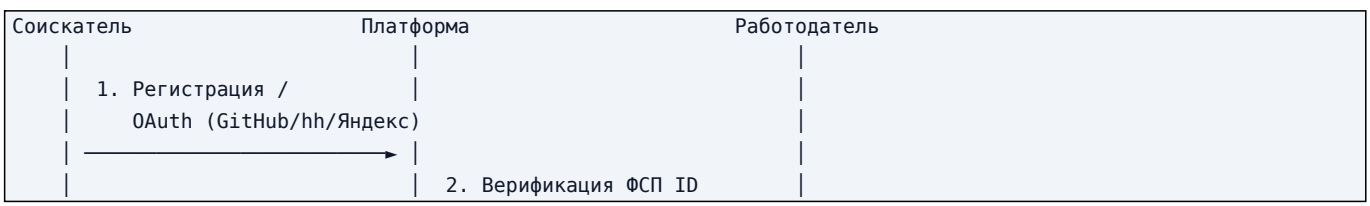
Платформа «ФСП Карьерная Арена» - это цифровая экосистема из трёх клиентских приложений (Web, Mobile, Swagger), общего бэкенда на Spring Boot 3 и базы данных PostgreSQL 16. Архитектура следует принципам Clean Architecture, Zero-Trust Security и полного соответствия 152-ФЗ РФ.

1.2. Компонентная схема





1.3. Потоки данных





1.4. Стек технологий

Слой	Технологии
Backend	Java 21 LTS, Kotlin 2.0, Spring Bo
Frontend	React 19, TypeScript 7, Tailwind C
Mobile	Kotlin 2.0, Jetpack Compose, Mater
БД	PostgreSQL 16 Alpine
Инфра	Docker, Docker Compose, Nginx (SPA
AI/LLM	Google Gemini 1.5 Flash, OpenRoute
Sandbox	Judge0 CE, Docker Container Engine
SSO (будущее)	Keycloak 24

---

2. Механика обратного найма и ранжирования

2.1. Концепция обратного найма (Reverse Recruiting)

В классическом найме соискатель откликается на вакансии. В «ФСП Карьерной Арене» всё наоборот:

1. Работодатель изучает верифицированный профиль спортсмена ФСП (ELO-рейтинг, спортивный разряд, стек технологий, ИИ-портрет).

2. Работодатель отправляет оффер с обязательной вилкой заработной платы (в рублях, «от-до»).

3. Соискатель принимает или отклоняет оффер. Контакты раскрываются только после принятия.

> Ключевой принцип: Работодатели конкурируют за таланты, а не наоборот.

2.2. Алгоритм матчинга

Ранжирование кандидатов для работодателя осуществляется по комплексной формуле CandidateMatchScore:

```
overallScore = w₁ × fspRankScore + w₂ × algoTestScore + w₃ × stackMatchScore + w₄ × experienceScore
```

Компонент	Вес	Описание
`fspRankScore`	35%	Спортивный разряд ФСП (ELO). Чем в

`algoTestScore`	25%	Результаты алгоритмического тестир
`stackMatchScore`	25%	Совпадение технологического стека
`experienceScore`	15%	Коммерческий опыт работы (годы, по

2.3. Система ELO-рейтинга и спортивных разрядов ФСП

Рейтинг ELO привязан к официальной системе ЕВСК Минспорта РФ по спортивному программированию:

Уровень	Разряд ФСП	ELO диапазон	Перцентиль	Требования
1	Квалификационный дивизион	1200 - 1399	Вне ТОПа	Регистрация в реестре ФСП
2	3-й спортивный разряд	1400 - 1699	ТОП-35%	1+ достижение, 5+ задач
3	2-й спортивный разряд	1700 - 1999	ТОП-15%	Призовое место, 25+ задач
4	1-й спортивный разряд	2000 - 2399	ТОП-5%	2+ достижения, 100+ задач
5	КМС (Кандидат в Мастера Спорта)	2400 - 2699	ТОП-1.5%	Победы на турнирах, 300+ задач
6	Мастер спорта РФ	2700 - 2999	ТОП-0.5%	1-е место ЧР, 500+ задач
7	Гроссмейстер / Чемпион РФ	3000 - 3500	ТОП-0.05%	Многократный чемпион, 800+ задач

2.4. Расчёт ELO

Формула расчёта ELO из ratingEngine.ts:

```
calculatedElo = 1200 (базовый старт)
              + algoScore (Easy×1 + Med×2.5 + Hard×6)
              + contestBonus ((contestRating - 1500) × 0.35)
              + arenaBonus (solvedTasks × 65)
```

При наличии контестного рейтинга ≥ 1600:

```
targetElo = 0.35 × targetElo + 0.65 × contestRating
```

ELO ограничен потолком 3000 для арены и 2950 для импортированных результатов LeetCode.

2.5. Определение грейда (Junior / Middle / Senior)

Грейд	Условия
Senior	ELO ≥ 2400 ИЛИ (опыт ≥ 5 лет И ELO
Middle	ELO ≥ 2000 ИЛИ (опыт ≥ 2 года И EL
Junior	Остальные случаи

2.6. Антифрод-защита ранжирования

Критически важное правило: кандидат с 0 достижениями и 0 решёнными задачами не может попасть в ТОП ни при каком ELO. Это защищает рейтинг от накрутки:

- Без истории ФСП → «Квалификационный дивизион (Без истории ФСП)»
- Есть ELO

2.7. Правила скрытия контактов (152-ФЗ)

Этап	Видимость контактов
Поиск кандидатов	Замаскированы: `+7 (9) *--42`, `@a
Оффер отправлен	Замаскированы
Оффер принят соискателем	Демаскируются - реальные данные
Каждый доступ	Запись в `security_audit_logs` (кт

2.8. «Слепой» наём (Anonymized / Blind Hiring Mode)

Для полного устранения когнитивных искажений и бессознательной предвзятости (Zero-Bias Standard) в платформу внедрен переключатель «Слепой» наем в панели работодателя:

- Маскирование идентификаторов: Имена кандидатов заменяются на стандартизированный псевдоним (например, `Атлет ФСП #0042 (Анонимный профиль)`), личные фотографии скрываются анонимной плашкой.

- Сохранен
достиже
задач на

---

3. Механика тестирования и защиты от «слива»

3.1. Алгоритмическая Арена ФСП

Арена - встроенная среда для решения алгоритмических задач с веб-IDE (Monaco Editor). Поддерживаемые языки: JavaScript, Python, Java, C++, Go.

Задачи импортируются из трёх источников:

1. Встроенный банк задач (классические LeetCode-задачи, адаптированные под ФСП)
2. Живой импорт из LeetCode через GraphQL прокси
3. Кастомные задания от работодателя (через AssignTestModal)

3.2. Расчёт ELO за решение задач

Сложность	ELO бонус	Пример
Easy	+50 ELO	Two Sum, Valid Parentheses
Medium	+80 ELO	LRU Cache, Merge Intervals
Hard	+100 ELO	Median of Two Sorted Arrays

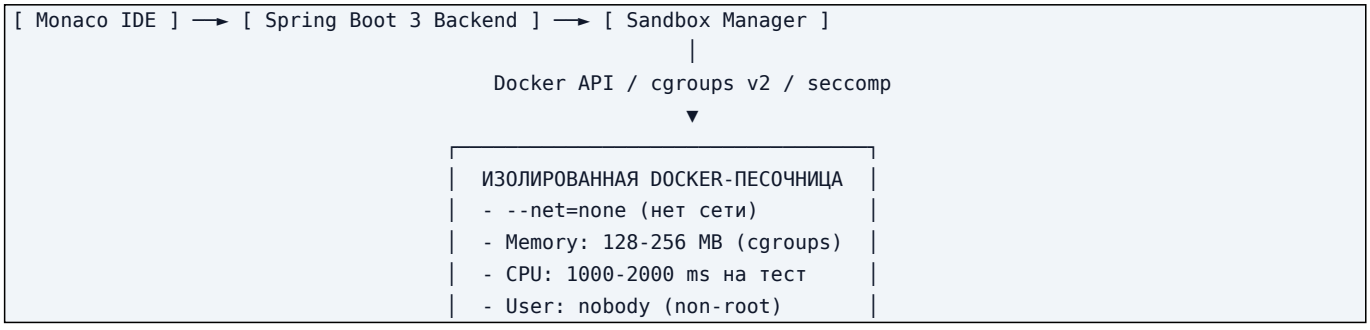
3.3. Ограничение кулдауна пересдачи

- Полная пересдача всех тестов возможна один раз в 3 месяца (поле `canRetakeAfterDate` в профиле кандидата).

- Отдельн

3.4. Изолированная Docker-песочница (Sandbox)

Каждая посылка кода выполняется в одноразовом контейнере с полной изоляцией:



```
                                    │  - FS: Read-Only rootfs          │
                                    │  - PIDs Limit: 32                │
                                    │  - seccomp: blocked syscalls     │
                                    └──────────────┬───────────────────┘
                                                   ▼
                                    [ Серверный Verdict Validator ]
                                    (ACCEPTED / WA / TLE / CE / RE)
```

3.5. Параметры изоляции

Параметр	Значение	Назначение
Лимит сессии	45 минут	Защита от растягивания
CPU Time	1000-2000 мс / тест	Отсечка неоптимальных O(N²) решени
RAM	128-256 MB	Защита от OOM хоста
Сеть	`--net=none`	Исключение curl / LLM / внешних БД
Файловая система	Read-Only + tmpfs 16MB	Защита от вредоносных скриптов
Процессы	PIDs ≤ 32	Защита от fork-бомб
Права	`nobody:65534`, `CAP_DROP_ALL`	Исключение эскалации привилегий

3.6. Антифрод-контроль

1. Скрытые тест-кейсы (Hidden Suites): Браузер получает только 2-3 открытых примера. Граничные и нагрузочные тесты (#3-#5) хранятся строго на бэкенде и инжектируются в контейнер.

2. Телеметрия честности:

- Отслеживание `window.blur` (потеря фокуса)
- Инспекци
- SHA-256 хэш исходного кода в таблице `task_submissions`
- Неизмен

---

4. Схема интеграции с ФСП ID

4.1. Текущая реализация

На текущем этапе профиль кандидата содержит:

- `fspId` - уникальный идентификатор в реестре ФСП РФ (например, `FSP-RUS-2026-0088`)
- `fspRank

4.2. Архитектурный подход к будущей интеграции с Keycloak

```
┌──────────────────────┐           ┌──────────────────────────┐
│  Пользователь        │           │  Keycloak 24             │
│  (Браузер / Mobile)  │ ◄──────►  │  Realm: fsp-arena        │
│                      │  OIDC     │                          │
│                      │  flow     │  Клиенты:                │
```



Шаги миграции:

1. Развернуть Keycloak 24 как отдельный сервис в docker-compose.yml
2. Создать Realm fsp-arena с ролями CANDIDATE, EMPLOYER, ADMIN
3. Настроить Identity Federation с реестром ФСП (fsp-russia.ru) через SAML/OIDC
4. Переключить Spring Security с собственного JWT на OAuth2 Resource Server с валидацией JWT от Keycloak
5. Маппировать keycloakSub на fspId при первой аутентификации

---

5. Спецификация API и безопасность

5.1. Swagger UI и OpenAPI

Ресурс	URL
Swagger UI (интерактивная документ	`http://localhost:8080/swagger-ui.
OpenAPI v3 JSON (машинная специфик	`http://localhost:8080/v3/api-docs
Облачный стенд	`https://linskay-demo-2026.hf.spac

5.2. Основные эндпоинты API (`/api/v1/`)

Метод	Эндпоинт	Описание	Роль
`POST`	`/auth/login`	Аутентификация (email + парол	Все → J

`POST`	`/auth/register`	Регистрация нового пользователя	Все
`GET`	`/auth/test-users`	Список тестовых аккаунтов для 1-кл	Все
`GET`	`/auth/privacy-policy`	Политика конфиденциальности (152-Ф	Все
`GET`	`/users/me`	Данные текущей сессии пользователя	Все
`PUT`	`/users/profile`	Обновление профиля пользователя	Все
`GET`	`/candidates`	Унифицированный поиск и фильтрация	EMPLOYER
`GET`	`/candidates/{id}/pdf`	Автогенерация официального PDF-пас	EMPLOYER
`GET`	`/vacancies/search`	Поиск вакансий с обязательной ФЗ-ВШК	CANDIDATE
`POST`	`/vacancies`	Создание вакансии	EMPLOYER
`POST`	`/sandbox/execute`	Выполнение кода в песочнице (Judge	CANDIDATE
`POST`	`/leetcode/fetch/{slug}`	Импорт задачи из LeetCode по slug	Все
`POST`	`/ai/generate-summary`	Мультимодальная генерация ИИ-портр	EMPLOYER
`POST`	`/ai/save-profile`	Сохранение ИИ-резюме в профиль кан	EMPLOYER
`POST`	`/offers`	Отправка оффера (с Idempotency-Key	EMPLOYER
`POST`	`/webhooks`	Регистрация исходящего веб-хука (Z	EMPLOYER
`GET`	`/webhooks/employer/{id}`	Список активных веб-хуков работода	EMPLOYER
`POST`	`/webhooks/{id}/ping`	Тестовая отправка пинга для провер	EMPLOYER
`POST`	`/files/upload`	Multipart загрузка резюме и доку	Все
`GET`	`/system/stats`	Открытая статистика платформы	Все
`POST`	`/auth/sync/github-profile`	Синхронизация решений из GitHub	CANDIDATE
`POST`	`/auth/sync/hh-resume`	Синхронизация резюме hh.ru	CANDIDATE
`GET`	`/audit/logs`	Журнал аудита безопасности (ФЗ-152	EMPLOYER

5.3. Ролевая модель (JWT)

```
Authorization: Bearer <JWT>
```

Роль	Описание	Разрешения
`CANDIDATE`	Соискатель / Спортсмен ФСП	Прохождение Арены, просмотр офферо
`EMPLOYER`	IT-компания / HR	Поиск кандидатов, создание ваканси

JWT содержит: userId, email, fullName, role, candidateId / employerId, fspElo, fspRank.

5.4. Идемпотентность API (RFC 7231)

При отправке оффера (POST /api/v1/offers):

1. Клиент генерирует заголовок Idempotency-Key: idem-<uuid>

2. Бэкенд проверяет таблицу api_idempotency_records:

- Первый запрос: Создаёт оффер, сохраняет ключ → `HTTP 201 Created`
- Повторны
  `Idempot

5.5. Соответствие 152-ФЗ

Мера	Реализация
Маскирование ПДН	Телефон, email, Telegram замаскиро
Согласие пользователя	Обязательный чекбокс при регистрац
Демаскировка по согласию	Контакты раскрываются только после

Журнал аудита	Таблица `security_audit_logs`: act
Доступ к аудиту	`GET /api/v1/audit/logs` + модальн

---

6. Инструкция по развёртыванию

6.1. Системные требования

Компонент	Минимум
Docker	24+
Docker Compose	v2+
RAM	4 GB
Диск	5 GB
ОС	Windows 10+, macOS 12+, Ubuntu 22.

6.2. Быстрый старт (одна команда)

Linux / macOS:

```
git clone https://github.com/linskay/fsp-2026.git
cd fsp-2026
chmod +x start.sh
./start.sh
```

Windows (cmd или PowerShell):

```
git clone https://github.com/linskay/fsp-2026.git
cd fsp-2026
start.bat
```

Или напрямую через Docker Compose:

```
docker compose up -d
```

6.3. Переменные окружения (.env)

Скопируйте .env.example в .env и при необходимости замените значения:

```
cp .env.example .env
```

Переменная	Описание	По умолчанию
`GEMINI_API_KEY`	API-ключ Google Gemini (ИИ-портрет	Предустановлен
`OPENROUTER_API_KEY`	API-ключ OpenRouter (мульти-модель	Предустановлен
`OLLAMA_API_URL`	URL локального Ollama сервиса	`http://localhost:11434/...`
`YANDEX_API_KEY`	API-ключ YandexGPT	(опционально)
`YANDEX_FOLDER_ID`	Folder ID Yandex Cloud	(опционально)
`APP_URL`	Адрес хостинга	`http://localhost:3000`
`GITHUB_CLIENT_ID`	GitHub OAuth2 Client ID	mock-fallback
`GITHUB_CLIENT_SECRET`	GitHub OAuth2 Client Secret	mock-fallback
`HH_CLIENT_ID`	hh.ru OAuth2 Client ID	mock-fallback
`HH_CLIENT_SECRET`	hh.ru OAuth2 Client Secret	mock-fallback
`JUDGE0_API_URL`	URL Judge0 Sandbox API	Предустановлен

6.4. Docker Compose сервисы

Сервис	Порт	Описание
`postgres`	5432	PostgreSQL 16 Alpine
`backend`	8080	Spring Boot 3 (Java 21)
`frontend`	3000	React 19 + Nginx
`ollama`	11434	Локальная LLM (опционально)

6.5. Проверка работоспособности

После запуска откройте:

1. Frontend: http://localhost:3000 - главная страница
2. Swagger UI: http://localhost:8080/swagger-ui.html - API документация
3. Healthcheck: curl http://localhost:8080/api/v1/auth/privacy-policy - статус бэкенда

6.6. Тестовые учётные записи

Роль	Email	Пароль
Кандидат	`candidate@fsp.ru`	`candidate2026`
Работодатель	`employer@fsp.ru`	`employer2026`

6.7. Остановка платформы

```
docker compose down          # Остановить контейнеры
docker compose down -v       # + удалить данные (volumes)
```

---

7. Команда проекта

NO PHP - NO PROBLEMS - 2026»

Проект разработан в рамках официального хакатона Федерации спортивного программирования России.

Зона ответственности	Описание
Архитектура & Backend	Spring Boot 3, миграции PostgreSQL
⚛ Frontend & UX/UI	React 19 + Vite, Code Arena IDE, и
Mobile App (Android)	Kotlin 2.0 + Jetpack Compose + Mat
☁ DevOps & AI Engine	CI/CD на GitHub Actions, мультипро

Манифест команды

1. Строгая статическая типизация - Java 21, Kotlin 2.0, TypeScript 7
2. Чистая архитектура - полное разделение слоёв во всех клиентах
3. Безопасность и закон - 152-ФЗ, маскирование ПДН, аудит
4. Инженерная эстетика - фирменный стиль ФСП, JetBrains Mono

> «NO PHP - NO PROBLEMS - 2026» - создаём цифровое будущее спортивного программирования России!